

AGROMIND
Community Project Report

Submitted by

RONGALI SRIVARUN (24331A4453)	NALLURI SANJANA (24331A4443)
CHINTAPALLI SINDU AKSHITHA (24331A4415)	SUBHOSRI MAHAPATRO (24331A4459)

In partial fulfillment for the award of the degree
of
BACHELOR OF TECHNOLOGY
IN

COMPUTER SCIENCE & ENGINEERING
(Data Science)

Under the esteemed Guidance of

Mrs. K.Amaravathi

Assistant Professor



DEPARTMENT OF DATA ENGINEERING
MAHARAJ VIJAYARAM GAJAPATHI RAJ COLLEGE OF
ENGINEERING (Autonomous)

(Approved by AICTE, New Delhi, and permanently affiliated to JNTUGV, Vizianagaram), Listed u/s 2(f) & 12(B)
of UGC Act 1956.

Vijayaram Nagar Campus, Chintalavalasa, Vizianagaram-535005, Andhra Pradesh
October, 2025

CERTIFICATE



This is to certify that the project entitled “AgroMind” is the bonafide work carried out by RONGALI SRIVARUN (24331A4453), NALLURI SANJANA (24331A4443), CHINTAPALLI SINDU AKSHITHA (24331A4415), and SUBHOSRI MAHAPATRO (24331A4459), of B.Tech V Sem CSE-DS, M.V.G.R. College of Engineering (Autonomous), Vizianagaram, during the year 2025-2026, in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology and that the project has not formed the basis for the award previously of any degree or any other similar title.

Signature of Project Guide

Mrs.K.Amaravathi
Assistant Professor
Data Engineering.

Signature of Head of the Department

Dr. V. Jyothi
Associate Professor
Data Engineering.

DECLARATION

We hereby declare that the work done on the dissertation entitled “**AgroMind**” has been carried out by us and submitted in partial fulfilment for the award of credits in Bachelor of Technology in Computer Science and Engineering (Data Science) of M.V.G.R College of Engineering (Autonomous) and affiliated to JNTUGV, Vizianagaram. The various contents incorporated in the dissertation have not been submitted for the award of any degree of any other institution or university.

RONGALI SRIVARUN
24331A4453

NALLURI SANJANA
24331A4443

CHINTAPALLI SINDU
AKSHITHA
24331A4415

SUBHOSRI MAHAPATRO
24331A4459

ACKNOWLEDGEMENT

We express our sincere gratitude to Mrs. K. Amaravathi for his/her invaluable guidance and support as our mentor throughout the project. His/Her unwavering commitment to excellence and constructive feedback motivated us to achieve our project goals. We are greatly indebted to him/her for his/her exceptional guidance.

Additionally, we extend our thanks to **Prof. P.S. Sitharama Raju (Director)**, **Dr. Y.M.C. Shekar (Principal)**, and **Dr. V. Jyothi (Head of the Department)** for their unwavering support and assistance, which were instrumental in the successful completion of the project. We are thankful for and fortunate enough to get constant encouragement and guidance from our Project Coordinator, **G.Gayathri**.

We also acknowledge the dedicated assistance provided by all the staff members in the Department of **Data Engineering**. Finally, we appreciate the contributions of all those who directly or indirectly contributed to the successful execution of this endeavor.

RONGALI SRIVARUN

24331A4453

NALLURI SANJANA

24331A4443

CHINTAPALLI SINDU AKSHITHA

24331A4415

SUBHOSRI MAHAPATRO

24331A4459

ABSTRACT

Agriculture depends strongly on timely irrigation, suitable soil conditions, and changing weather conditions. Farmers may have access to soil-moisture information but can face difficulty in converting soil and weather conditions into clear irrigation decisions. **AgroMind Smart Farming** is a web-based smart irrigation and farm advisory system developed to provide simple and practical irrigation guidance. The system considers factors such as crop type, soil type, farm size, location, water availability, and soil moisture, along with weather conditions. This approach is consistent with established irrigation-planning methods that consider crop, soil, and climate information.

The system allows farmers to create and manage farm profiles and enter soil-moisture values directly or provide multiple readings, which are averaged to obtain a representative value. Weather information such as **temperature, humidity, rainfall, and rain probability** is obtained automatically using the Open-Meteo service. The Irrigation Planner analyzes the available information using predefined decision rules and generates irrigation status, recommended time, duration, estimated water requirement, risk level, and water-saving suggestions. The Dashboard, Weather, Crop Health, and Analytics modules provide a clear view of the farm condition and historical information.

AgroMind also includes an **AI Assistant** that uses the available farm context to explain irrigation results and answer natural-language farming questions. The system is developed using **HTML, CSS, JavaScript, Flask, PostgreSQL, Open-Meteo, and AI-assisted services**. The overall workflow consists of collecting farm information, obtaining weather data, processing soil-moisture readings, analyzing the conditions, generating an irrigation recommendation, and providing an understandable explanation to the farmer. The proposed system aims to support practical irrigation planning and promote more informed water-management decisions through an accessible web-based platform.

KEYWORDS: Smart Irrigation, Artificial Intelligence, Precision Agriculture, Soil Moisture Analysis, Water Management, Crop Management, Weather Data, Irrigation Requirement Analysis, AI-Assisted Analysis, Open-Meteo API, Python Flask, PostgreSQL, Data Visualization, Agricultural Advisory System.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DB	Database
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JS	JavaScript
JSON	JavaScript Object Notation
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UX	User Experience
URL	Uniform Resource Locator
ML	Machine Learning
PWA	Progressive Web Application
GPS	Global Positioning System
FAO	Food and Agriculture Organization

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1.1	System flow	17
Figure 1.2	System architecture	18
Figure 1.3	Login page	24
Figure 1.4	Farm management	25
Figure 1.5	Dash board	26
Figure 1.6	Planner 1	27
Figure 1.7	Planner 2	28
Figure 1.8	weather	29
Figure 1.9	Crop health	30
Figure 1.10	Analytics 1	31
Figure 2.1	Analytics 2	31
Figure 2.2	AI assistant	32

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Farm data	13
Table 1.2	Soil-moisture data	13
Table 1.3	Weather data	14
Table 1.4	Challenges faced	31
Table 1.5	TechStack	35
Table 1.6	Application structure	36

TABLE OF CONTENTS

Contents	Page No.
Abstract	5
List of Abbreviations	6
List of Figures	7
List of Tables	8
1. Introduction	10
1.1 Problem Statement	10
1.2 Project Objective	11
1.3 Scope of the Project	11
2. Literature Survey	12
3. Data Gathering / Data Used	13-14
4. Methodology / System Design	15-18
5. Implementation / Modules	19-22
6. Results / Outputs	23-28
7. Impact Assessment	29-30
8. Challenges Faced	31
9. Conclusion	32
10. Future Work	33
References	34
Appendix A: Packages, Tools Used & Working Process	35
Appendix B: Source Code	36-41
Project Documentation Status	42

1. INTRODUCTION

Agriculture is an important sector that depends heavily on effective water management. Proper irrigation requires consideration of factors such as crop type, soil characteristics, soil moisture, weather conditions, and water availability. Tools such as FAO CROPWAT demonstrate the importance of using soil, climate, and crop information for irrigation planning and scheduling.

Small and community farmers may often depend on fixed schedules or personal experience to decide when to irrigate their crops. These methods may not always account for changing soil-moisture and weather conditions, which can result in over-irrigation, under-irrigation, or unnecessary water usage. Therefore, there is a need for a simple and accessible system that can combine farm information, manual soil-moisture observations, and weather data to support better irrigation decisions.

AgroMind Smart Farming is a web-based agricultural decision-support system developed to provide simple and understandable irrigation guidance. The system uses farm details, soil-moisture readings, and automatically obtained weather information to analyze irrigation requirements using predefined rules. It also provides Dashboard, Irrigation Planner, Weather, Crop Health, Analytics, and AI Assistant modules. The Generative AI assistant helps explain recommendations and answer farming-related questions, while the system is designed to work without requiring permanent IoT infrastructure.

1.1 Problem Statement

Small and community farmers often face difficulty in deciding **when to irrigate and how much water to use**. Irrigation decisions based mainly on fixed schedules or personal experience may not sufficiently consider soil moisture, crop requirements, and changing weather conditions, which can lead to water wastage or inadequate irrigation. Established irrigation approaches also consider crop, soil, and climate conditions for scheduling.

Many advanced smart-farming solutions depend on sensors, IoT devices, or complex infrastructure, which may not be suitable for every small-scale farming environment. Therefore, there is a need for a **simple, affordable, and user-friendly web-based system** that can combine farm information, manual soil-moisture readings, and weather data to provide understandable irrigation guidance.

1.2 Project Objective

- **Farm Profile & Weather:** Maintain a simple digital farm profile covering crop, soil, location, farm size, and water availability, along with current weather information relevant to farming decisions.
- **Soil-Moisture Input:** Accept a known soil-moisture value or calculate an average from multiple readings.
- **Irrigation Decision Engine:** Analyze farm and weather conditions to give the irrigation requirement, timing, duration, estimated water quantity, risk level, and water-saving advice.
- **Crop Health & Analytics:** Summarize crop health using moisture, temperature, and humidity, and provide historical analytics on soil moisture, temperature, water use, and irrigation activity.
- **Farm-Aware AI Assistant:** Explain recommendations and answer farming questions using the farm's own context.

1.3 Scope of the Project

- **Farm Management**
Allows farmers to create, save, and manage multiple farm profiles with details such as location, crop, soil type, farm size, and water availability.
- **Soil-Moisture Management**
Allows users to enter a known soil-moisture value or multiple readings and calculates the average when required.
- **Weather Integration**
Automatically obtains important weather information such as temperature, humidity, rainfall, and rain probability for the selected farm location.
- **Irrigation Decision Support**
Analyzes farm, soil-moisture, and weather information using predefined agricultural rules to provide irrigation recommendations.
- **AI-Assisted Farming Guidance**
Provides farmer-friendly explanations, suggestions, and answers to agriculture-related questions through the AI Assistant.
- **Farm Monitoring and Records**
Provides Dashboard, Crop Health, and Analytics modules to display current conditions and maintain previous farm analysis and irrigation records.

2. LITERATURE SURVEY

2.1 Crop Water Requirement and Irrigation Scheduling

Irrigation scheduling is an important part of agricultural water management. The irrigation requirement of a crop depends on factors such as **crop characteristics, soil conditions, weather, rainfall, and available soil water**. FAO CROPWAT is a decision-support tool that calculates crop-water and irrigation requirements using soil, climate, and crop data and can develop irrigation schedules based on soil-water balance.

2.2 Agricultural Decision Support Systems

Agricultural Decision Support Systems (DSS) help farmers and agricultural professionals make decisions by combining different types of agricultural information. Models such as FAO AquaCrop use **weather data, crop characteristics, soil characteristics, and management practices** to study crop response to water and support irrigation and water-management decisions.

2.3 Weather and Soil Information for Irrigation

Weather and soil conditions play an important role in determining irrigation requirements. Temperature, rainfall, and other climatic variables influence crop water use, while soil characteristics affect the amount of water available to crops. Recent irrigation-scheduling guidance also emphasizes considering **crop water requirements, soil-water relationships, climate, and crop growth stages** when planning irrigation.

2.4 AI and Machine Learning in Agriculture

Artificial Intelligence and Machine Learning techniques are increasingly used in agriculture for applications such as **crop monitoring, disease detection, yield prediction, and irrigation management**. AI-based systems can process historical and environmental data to identify patterns and provide recommendations. In addition, AI assistants can present agricultural information in a simpler and more interactive form for users.

2.5 Research Gap

Existing agricultural decision-support approaches show that effective irrigation planning requires the consideration of **crop, soil, weather, and water-related information**. However, many existing tools are designed for detailed agricultural modelling and may require specialized knowledge to operation.

3. DATA GATHERING / DATA USED

3.1 Farm Data

Agromind uses a farm profile as the persistent context for its modules. The farmer supplies crop, soil type, farm size, location, and water availability. These values are reused by the Dashboard, Irrigation Planner, Crop Health module, Analytics, and AI Assistant.

Table 1.1: Farm Data

Data Item	Description	Primary Use
Crop	Selected crop grown in the farm	Irrigation and crop-care context
Soil Type	Selected soil category	Irrigation interpretation
Farm Size	Area of the farm	Estimated water quantity
Location	Farmer-selected location	Weather retrieval
Water Availability	Available irrigation water condition	Water-risk recommendation
Growth Context	Farm/crop stage when provided	Advisory context

3.2 Soil-Moisture Data

The Irrigation Planner supports two input modes. In the first mode, the farmer enters a known soil-moisture value directly, such as 25%. In the second mode, the farmer enters several readings. The system calculates their average and uses the resulting value as the final soil-moisture input.

Table 1.2: Soil-Moisture Data

Mode	Example	Processing
Known value	25%	Use the entered value directly
Multiple readings	22%, 25%, 24%, 23%, 26%, 25%	Calculate arithmetic mean = 24.17%

The multi-reading option is useful when the farmer has taken readings from different points or at different nearby observations and wants a representative value for the planning calculation. The average is treated as an input to the decision engine; it is not presented as a replacement for professional field measurement.

3.3 Weather Data

Weather information is automatically retrieved after the farmer selects or provides a location. AgroMind uses Open-Meteo as the weather-data source in the proposed technology flow. The Weather module presents current temperature, humidity, rain probability, rainfall, weather condition, and a three-day forecast. The same relevant weather values are made available to the Irrigation Planner.

Table 1.3: Weather Data

Weather Variable	Use in AgroMind
Temperature	Helps interpret current crop and water-demand conditions.
Humidity	Supports moisture-condition interpretation.
Rain Probability	Helps identify whether irrigation may be postponed.
Rainfall	Provides recent/current precipitation context.
Weather Condition	Gives a simple human-readable status.
3-Day Forecast	Supports short-term planning and warnings.

3.4 Historical and Activity Data

Analytics uses stored farm observations and irrigation activity to show total analyses, average soil moisture, total water used, water saved, soil-moisture trends, temperature trends, water-usage charts, irrigation activity, and a historical analysis table. These records provide a view of how farm conditions and irrigation decisions change over time.

3.5 Data Preparation

Data preparation in AgroMind is primarily validation, averaging, formatting, and contextual organization rather than a machine-learning training pipeline. Soil-moisture readings are treated as percentages, multiple readings are averaged arithmetically, weather responses are mapped to the fields required by the interface, and farm profile values are retained as context. This keeps the prototype's core recommendation path understandable and easy to demonstrate.

4. METHODOLOGY / SYSTEM DESIGN

The methodology of AgroMind Smart Farming follows a step-by-step process in which farm information, soil-moisture readings, and weather conditions are collected and processed to provide irrigation guidance.

1. User Login and Farm Selection

The farmer first logs into the system and creates or selects a farm profile. The farm profile contains important information such as farm name, location, crop type, soil type, farm size, and water availability.

2. Soil-Moisture Collection

The farmer provides the soil-moisture information. If a reliable value is already available, it can be entered directly. If multiple readings are taken from different locations in the farm, the system calculates their average to obtain a representative soil-moisture value.

3. Weather Data Collection

The system obtains weather information automatically based on the selected farm location. AgroMind uses the Open-Meteo Weather API, which provides variables such as temperature, relative humidity, precipitation, and precipitation probability.

4. Data Processing and Analysis

The collected farm information, soil-moisture value, and weather conditions are passed to the irrigation decision engine. The system checks these inputs against the predefined rules used by the project.

5. Irrigation Decision

The decision engine determines whether irrigation is required based on the available conditions. If irrigation is recommended, the system provides information such as irrigation status, suggested time, duration, estimated water quantity, risk level, and water-saving suggestions.

6. AI-Assisted Explanation

The AI Assistant uses the available farm context and the generated result to provide a simple explanation. It can also answer natural-language farming questions and provide general guidance.

7.Storage and Monitoring

The farm information and analysis results are stored in the PostgreSQL database. Historical records are used by the Analytics module to help the farmer monitor changes in soil moisture, weather conditions, and irrigation activities.

8.Dashboard Output

The final information is presented through the Dashboard and other modules such as Irrigation Planner, Weather, Crop Health, Analytics, and AI Assistant.

Overall Methodology Flow

Farm Details → Soil Moisture → Weather Data → Data Processing → Decision Engine → Irrigation Recommendation → AI Explanation → Database → Dashboard

4.1 System Design

The System Flow begins with the farmer logging in and creating or selecting a farm profile.

The farmer then enters crop, soil, farm size, location, and water availability, followed by soil moisture readings.

The system fetches real-time weather data using the Open-Meteo API and analyzes the farm, soil, and weather conditions.

Based on this analysis, the system generates irrigation recommendations, including timing, water quantity, and risk level.

Finally, the AI Assistant provides explanations and answers, while the dashboard displays all relevant farm details and recommendations.

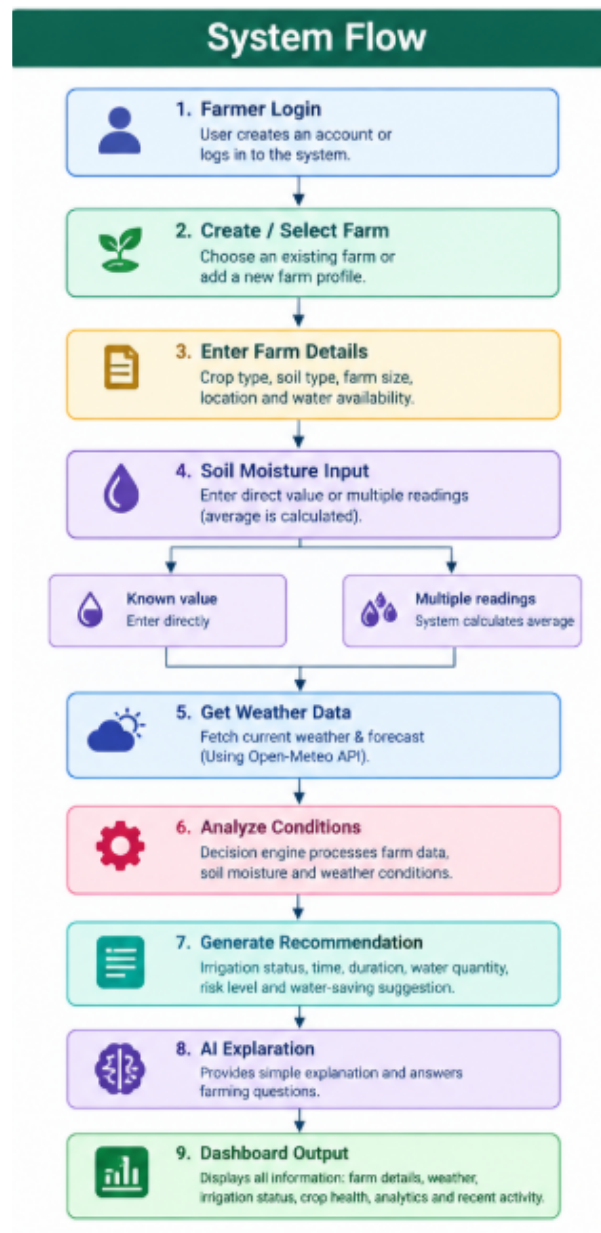


Figure 1.1: System Flow

4.2 SYSTEM ARCHITECTURE

The System Architecture consists of a web-based frontend developed using HTML, CSS, and JavaScript, through which users can enter farm and soil details.

The Flask backend handles API requests, business logic, decision-making, and AI integration.

PostgreSQL stores farm data, soil moisture records, weather data, historical analytics, and user information.

External services such as Open-Meteo API provide weather data, while the AI Assistant supports explanation and recommendation generation.

The overall flow processes farm data → weather data → soil moisture → analysis → irrigation recommendation and explanation.

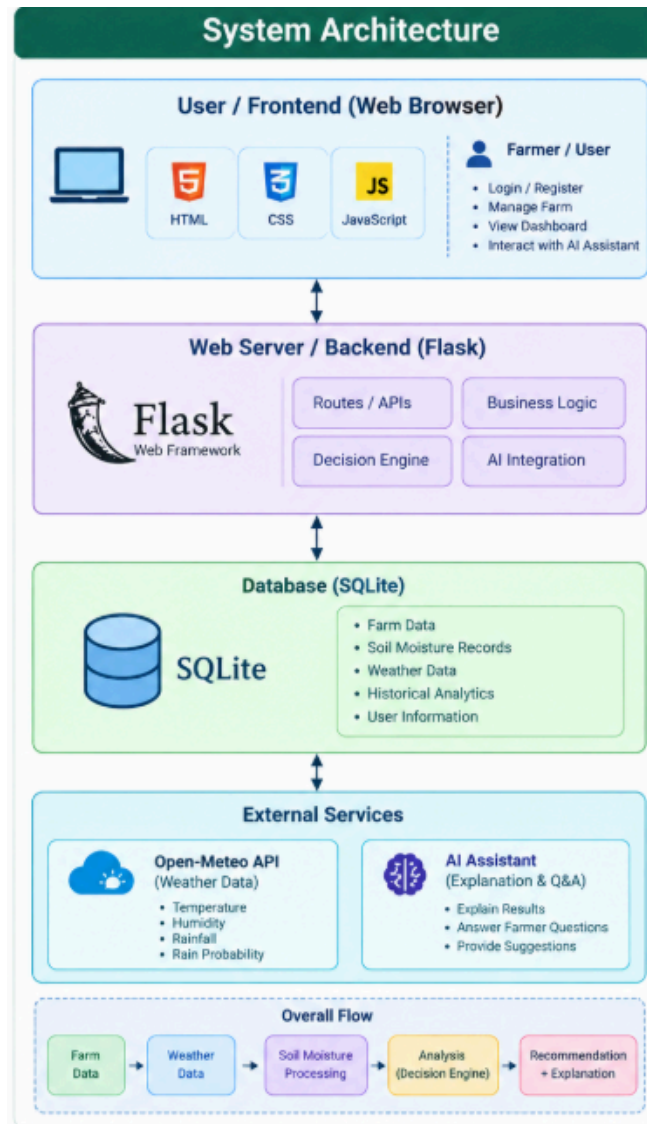


Figure 1.2: System Architecture

5. IMPLEMENTATION / MODULES

5.1 Implementation

AgroMind Smart Farming is implemented as a web-based agricultural decision-support system. The frontend is developed using HTML, CSS, and JavaScript, while Flask/Python is used for backend processing. PostgreSQL is used to store user, farm, analysis, and historical information.

The implementation follows a modular approach in which each module performs a specific function and communicates with the backend when required. Weather information is obtained from Open-Meteo using the farm's location. Open-Meteo provides current and forecast weather variables including temperature, relative humidity, precipitation, and precipitation probability.

The irrigation analysis is performed by a rule-based decision engine. It receives farm details, soil-moisture information, and weather conditions as inputs and generates an irrigation recommendation. The AI Assistant is used separately to explain the generated results and answer farming-related questions.

5.2 Modules

1. User Authentication and Farm Management

This module manages user login and farm profiles. A farmer can create and save a farm by entering:

- Farm name
- Location
- Crop type
- Soil type
- Farm size
- Water availability

A user can manage multiple farms and select the required farm for analysis.

2. Dashboard Module

The Dashboard provides a quick overview of the selected farm. It displays:

- Soil moisture
- Temperature
- Humidity
- Rainfall

- Rain probability
- Irrigation status
- Crop health
- Farm information
- Recent activities

This allows the farmer to understand the current farm condition without opening every module.

3. Irrigation Planner Module

The Irrigation Planner is the main decision-support module. It combines:

- Crop type
- Soil type
- Farm size
- Water availability
- Soil moisture
- Temperature
- Humidity
- Rainfall
- Rain probability

The collected values are processed by the irrigation decision engine, which generates an irrigation recommendation.

4. Soil-Moisture Processing Module

This module handles manual soil-moisture readings.

Two methods are supported:

1. Direct Measurement

Known Soil Moisture → Enter Measured Value

2. Multiple Measurements

Reading 1 → Reading 2 → Reading 3 → Reading 4 → Reading 5

↓

Average Moisture Calculation

↓

Representative Soil Moisture Value

The calculated value is then passed to the irrigation decision engine.

5. Weather Module

The Weather module obtains weather information for the selected farm location using Open-Meteo. The API supports current conditions and forecast data, including temperature, humidity, precipitation, and precipitation probability.

The module displays:

- Current temperature
- Humidity
- Rainfall/precipitation
- Rain probability
- Short-term forecast

6. Irrigation Decision Engine

This is the core processing component of the system.

The engine receives:

Farm Data + Soil Moisture + Weather Data

and applies predefined project rules to determine:

- Whether irrigation is required
- Suggested irrigation time
- Suggested duration
- Estimated water requirement
- Risk level
- Water-saving recommendation

The engine provides the actual irrigation decision, while the AI module can explain that decision in simpler language.

7. Crop Health Module

The Crop Health module provides a simplified indication of the current crop condition based on available farm and environmental information.

It considers parameters such as:

- Soil moisture
- Temperature
- Humidity

- Recent environmental conditions

The module presents the result in an easy-to-understand form for the farmer.

8. Analytics Module

The Analytics module maintains historical information and presents it using charts and summaries.

It can show:

- Soil-moisture trends
- Temperature trends
- Irrigation records
- Water-use information
- Previous analysis results

This helps the farmer observe changes in farm conditions over time.

9. AI Assistant Module

The AI Assistant provides a conversational interface for farmers. It uses the available farm context and irrigation results to:

- Explain irrigation recommendations
- Answer farming-related questions
- Provide general suggestions
- Explain weather or soil conditions
- Help the farmer understand system results

The AI Assistant does not replace the rule-based irrigation decision engine; it primarily provides explanation and interactive guidance.

10. Database Module

PostgreSQL is used for persistent data storage. The database stores information such as:

- User details
- Farm profiles
- Soil-moisture readings
- Irrigation analysis
- Historical records
- AI conversations

6. RESULTS / OUTPUTS

The AgroMind system produces the following outputs through its different screens and modules.

6.1 Login Page

The Login Page allows registered users to securely access their AgroMind account and their saved farm information.

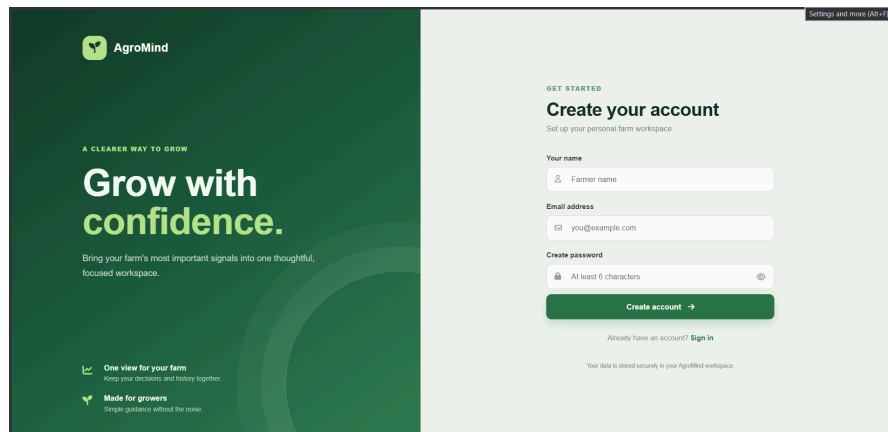


Figure1.3: Login Page

6.2 Farm Management

The farmer can create, save, select, edit, and manage farm profiles containing crop, soil, location, farm size, and water availability details.

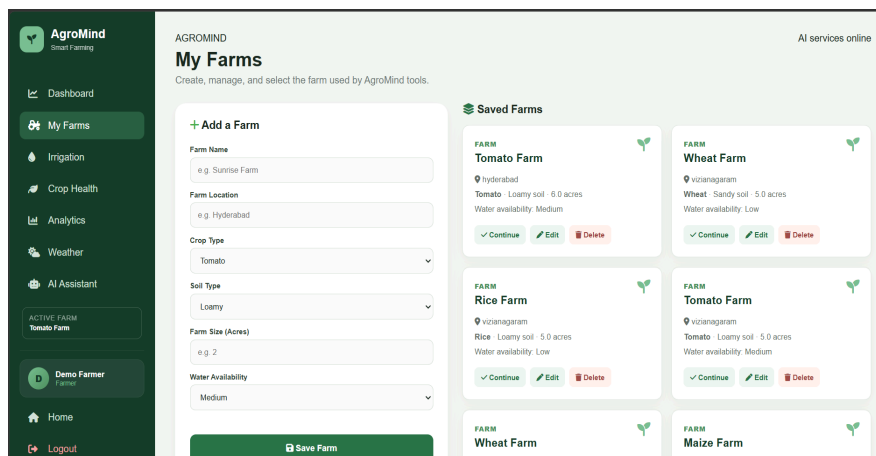


Figure 1.4: Farm Management

6.3 Dashboard

The Dashboard provides a quick summary of:

- Soil moisture
- Temperature
- Humidity
- Rainfall
- Rain probability
- Irrigation status
- Crop health
- Recent activities

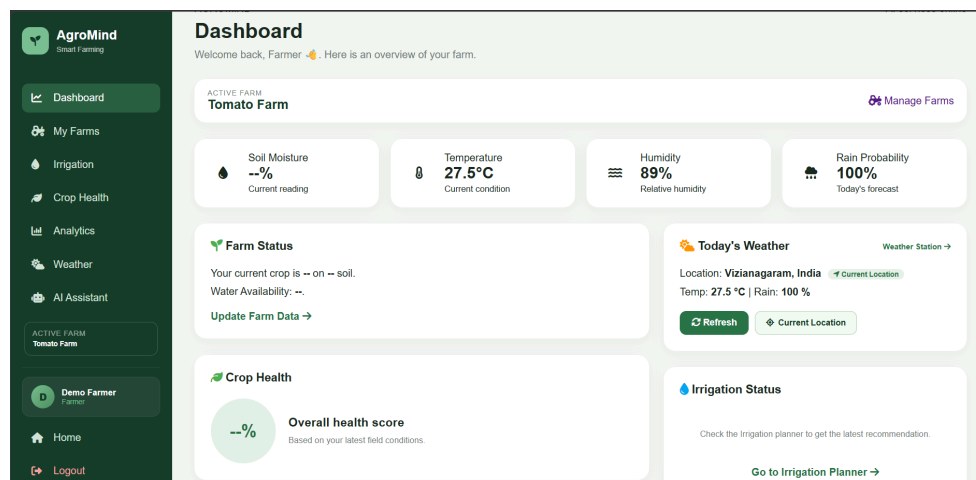


Figure 1.5: DashBoard

6.4 Irrigation Planner

The system analyzes farm, soil-moisture, and weather information and displays:

- Irrigation required / not required
- Recommended irrigation time
- Duration
- Estimated water requirement
- Risk level
- Water-saving suggestion

STEP 1 — LOCATION

Farm Location & Live Weather

📍 Farm Location
Hyderabad

🌤️ Fetch Weather

🟢 Live weather loaded for Hyderabad, India

29.7
Temperature (°C)

61
Humidity (%)

82
Rain Probability (%)

STEP 2 — SOIL MOISTURE

How well do you know your soil moisture?

☒ I know my soil moisture
 ☐ ? I don't know my soil moisture

Enter readings from up to 6 spots in your field. At least 5 valid readings are required. Leave a field blank to skip it.

Reading 1 21	Reading 2 25	Reading 3 25
Reading 4 51	Reading 5 20.2	Reading 6 (optional) e.g. 27

📊 Calculated Average Moisture: **28.4%** (5 readings)

Figure 1.6: Planner1

STEP 3 — FARM DETAILS

Crop & Field Information

🌾 Crop Type: Rice

 🏔️ Soil Type: Loamy

📏 Farm Size (Acres): 2

 💧 Water Availability: Medium

Analyze & Get Recommendation

Irrigation Not Required

Why: Delay irrigation because the field already has 28.4% soil moisture and rain is likely soon.
 Main condition: Rain probability is 82%, so the upcoming rain should provide sufficient water.
 Water-saving tip: Keep the irrigation lines turned off. Turn them on only if the rain does not arrive or the soil dries out sharply.
 Caution: Watch the forecast. If the rain stalls or humidity drops, reassess the crop for water stress.

Soil moisture used: 28.4% (average of your readings) | Temperature: 29.7°C | Rain probability: 82%

Figure 1.7: Planner2

6.5 Weather

The Weather module displays current weather and forecast information for the selected farm location. Open-Meteo provides variables such as temperature, humidity, precipitation, and precipitation probability.

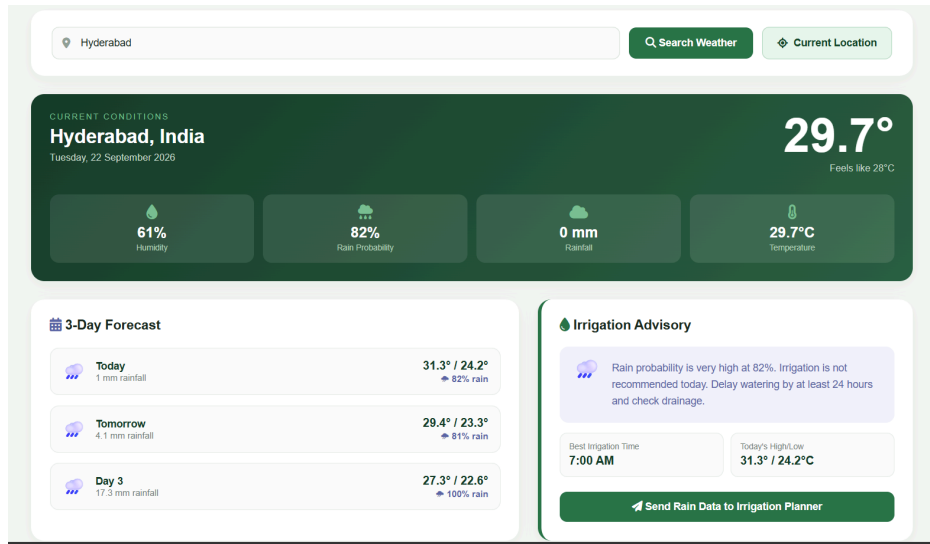


Figure 1.8:weather

6.6 Crop Health

The Crop Health module provides a simple crop-health indication based on available soil and environmental conditions.

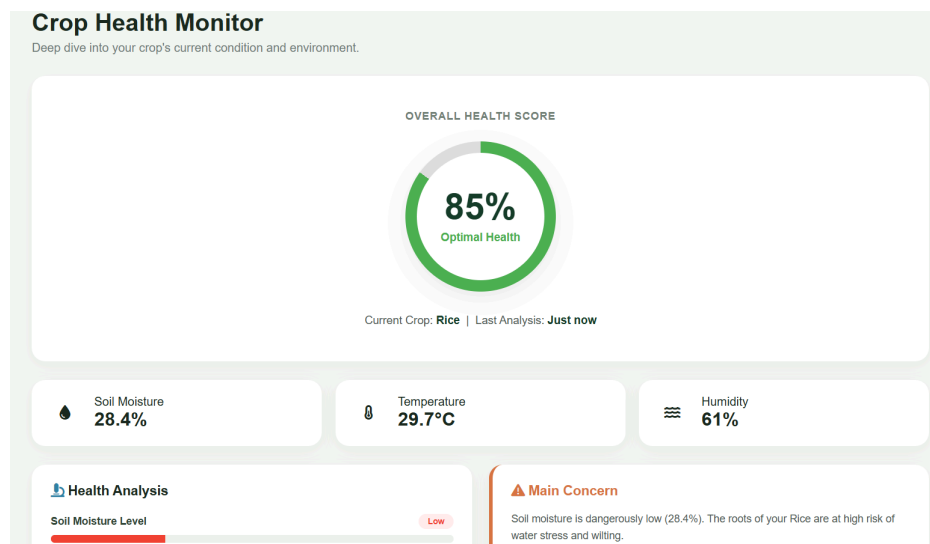


Figure 1.9: Crop Health

6.7 Analytics

The Analytics module displays historical information through charts, including soil-moisture trends, irrigation records, and water usage.

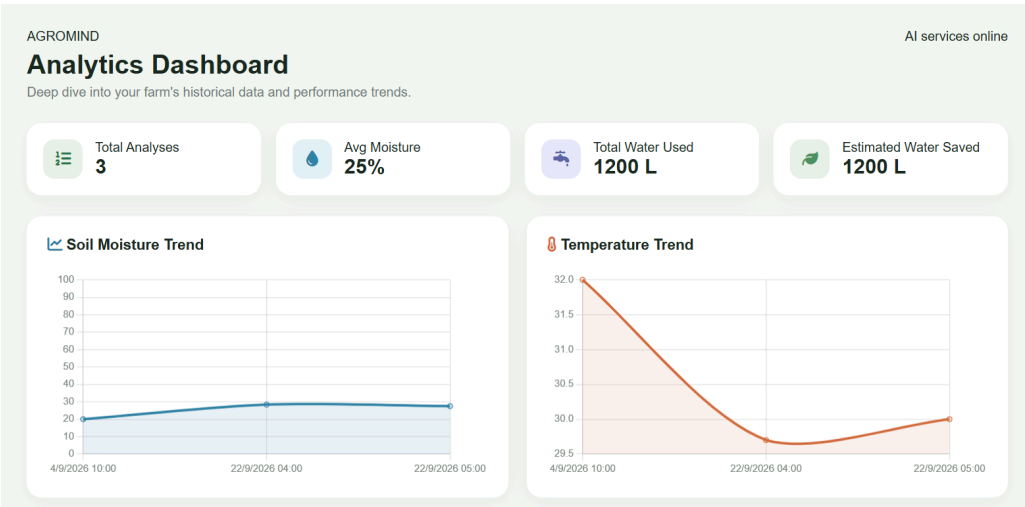


Figure 1.10: Analytics1

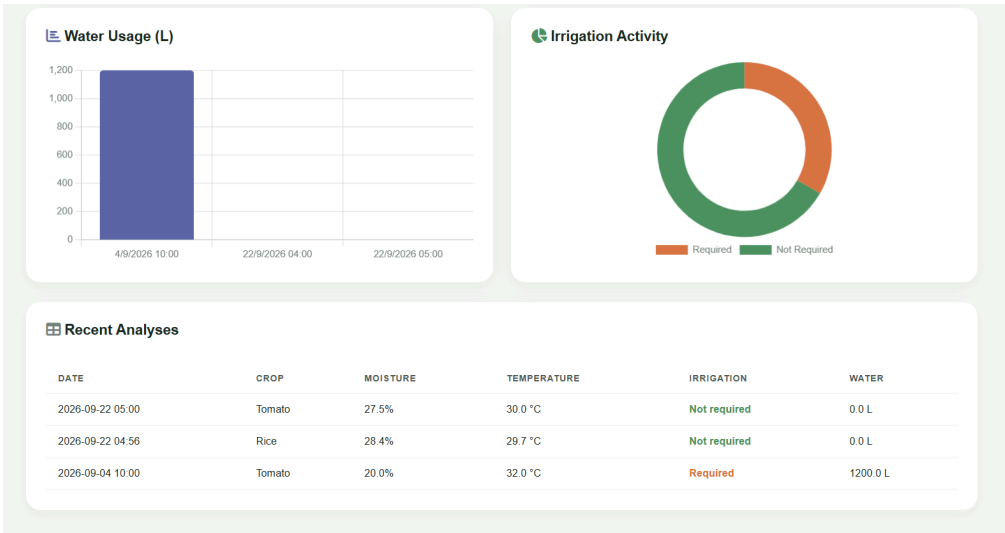


Figure 2.1: Analytics 2

6.8 AI Assistant

The AI Assistant explains irrigation results and allows the farmer to ask agriculture-related questions using natural language.

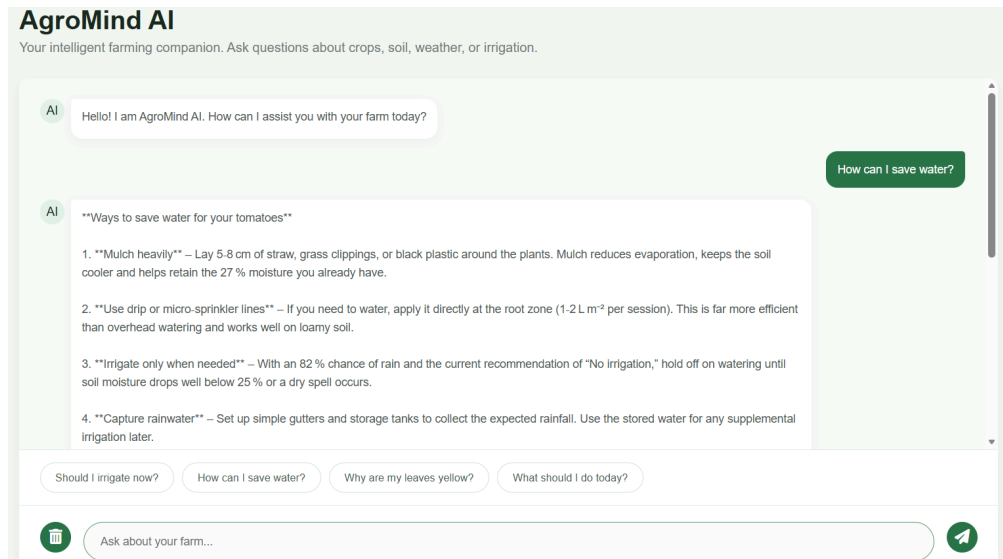


Figure 2.2: AI-Assistant

6.9 Overall Output

Login → Farm Selection → Farm Details → Soil Moisture → Weather → Irrigation Analysis → Recommendation → AI Explanation → Dashboard & Analytics

The system successfully presents farm information and irrigation guidance in a simple and user-friendly web interface.

7. IMPACT ASSESSMENT

7.1 Farmer Impact

AgroMind is intended to reduce the effort required to interpret several farm conditions at once. Instead of manually comparing soil moisture, weather, crop, soil, and water availability, the farmer receives a consolidated recommendation. The simple dashboard and dedicated planner can improve accessibility for users who prefer clear actions over complex analytics.

7.2 Water-Use Impact

The system emphasizes irrigation only when the analyzed conditions indicate a need. Weather-aware recommendations can help avoid unnecessary watering when rainfall is likely, while water-saving advice encourages appropriate timing and controlled use. The project does not claim a measured percentage of water savings without field validation; the water-saving value should be evaluated using real farm records.

7.3 Decision-Making Impact

A transparent decision path can improve the farmer's understanding of why a recommendation was generated. The planner exposes the main inputs and the final decision, while Analytics allows the farmer to compare current conditions with historical records. This can support more consistent planning than relying only on memory or fixed schedules.

7.4 Educational Impact

The project demonstrates how web development, databases, external APIs, decision logic, data visualization, and AI can be combined to solve a practical agricultural problem. It also provides a foundation for future student work involving real sensors, validated agronomic models, and field testing.

7.5 Operational Feasibility

AgroMind is designed as a browser-based application, which reduces the hardware requirements for the prototype. A farmer can interact with the system through a web interface, while weather information is retrieved automatically. The architecture separates external services from the decision logic, making it possible to replace or extend components later.

7.6 Environmental Relevance

Efficient irrigation can contribute to responsible water use when recommendations are accurate and followed appropriately. AgroMind supports this objective by considering rainfall and soil moisture before suggesting irrigation. The project does not claim direct environmental

measurements; its contribution at prototype stage is the provision of a water-aware decision-support workflow.

7.7 Scalability

The modular design can be extended from a single farm profile to multiple farms and users. Additional crops, soil categories, validated moisture references, richer weather variables, and sensor inputs can be added without changing the basic navigation structure. The Flask backend and PostgreSQL prototype can also be replaced or scaled with a larger relational database when deployment requirements increase.

7.8 Limitations of Impact Assessment

The current project is a prototype and its impact should be treated as expected or potential impact rather than measured field performance. Real-world validation requires long-term farm observations, reliable sensor measurements, crop-specific reference data, comparison with existing irrigation practice, and assessment across different seasons and locations.

8. CHALLENGES FACED

Table 1.4: Challenges

Challenge	Project Response
Soil moisture may be unavailable	Provide direct value input and multi-reading average option.
Weather changes continuously	Use automatic Open-Meteo data for the selected location.
Many inputs can confuse users	Separate Dashboard, Weather, Planner, Health, Analytics, and AI modules.
Recommendation must be understandable	Use structured outputs: requirement, time, duration, quantity, risk.
AI should not replace decision logic	Generate a structured result first, then use AI for explanation.
Historical data may be limited	Design Analytics around stored activity and observations.
External API failure	Keep validation/error handling around weather and AI requests.
Different farm conditions	Keep crop, soil, farm size, location, and water availability configurable.
Security of service keys	Keep external API credentials on the server side.
Overclaiming agronomic accuracy	Present the system as decision support and identify validation needs.

8.1 Usability Challenge

One of the main design challenges is balancing technical detail with simplicity. A farmer may not want to interpret several charts before deciding whether to irrigate. AgroMind therefore uses progressive detail: the Dashboard provides the summary, the Planner provides the calculation and recommendation, Weather provides detailed forecast information, and Analytics provides historical evidence.

8.2 Data Quality Challenge

The quality of an irrigation recommendation depends on the quality of soil-moisture readings and other inputs. The multiple-reading feature helps the farmer obtain a representative average, but averaging cannot correct an incorrect or poorly collected measurement. Future versions should support validated sensor readings and field calibration.

9. CONCLUSION

AgroMind is a web-based smart irrigation and farm advisory system designed to help farmers understand current farm conditions and make informed irrigation decisions. It integrates a farm profile, current weather information, soil-moisture input, irrigation decision logic, crop health summary, historical analytics, and a farm-aware AI Assistant into a single platform. The central workflow involves entering farm details, selecting a location, fetching weather data, entering soil-moisture values directly or calculating them from multiple readings, analyzing the combined conditions through the decision engine, and generating a clear irrigation recommendation, while AI provides a natural-language explanation of the result. The proposed website structure addresses the major objectives of the project by providing a farmer profile, dashboard, weather module, irrigation planner, crop health module, analytics, and AI Assistant, with a modular design that connects these features into one integrated workflow rather than isolated components. However, the system has certain limitations: its recommendations are decision-support outputs that require real-world validation; weather and AI functionality depend on external services and network availability; soil-moisture accuracy depends on the quality of farmer-provided readings or future sensor input; the prototype does not directly control physical irrigation equipment; and the actual impact and water-saving benefits should be measured through field trials rather than assumed.

10. FUTURE WORK

10.1 Real-Time Soil Sensors

Future versions can integrate soil-moisture sensors so that readings are collected automatically instead of being entered manually. Multiple sensor points could provide spatial information and reduce dependence on a single measurement.

10.2 Automated Irrigation Hardware

After sufficient field validation, AgroMind could be connected to pumps, valves, or irrigation controllers. Such integration should use safety limits, manual override, sensor validation, and fail-safe behavior so that a software recommendation cannot unintentionally cause excessive watering.

10.3 Advanced Crop Models

Crop-specific and soil-specific reference datasets can be expanded and validated with agricultural expertise. More detailed models could consider crop growth stage, evapotranspiration, root-zone conditions, and irrigation method.

10.4 Improved Weather Intelligence

Future work can extend the weather module with longer forecasts, additional precipitation variables, severe-weather alerts, and localized farming advisories. Historical weather can also be combined with irrigation records for improved planning.

10.5 Advanced Analytics

Analytics can be extended with predictive trends, comparative water-use reports, farm-to-farm benchmarking, seasonal summaries, and exportable reports. These features can help farmers evaluate whether irrigation practices are becoming more efficient.

10.6 AI Improvements

The AI Assistant can be expanded with retrieval from validated agricultural knowledge sources, multilingual interaction including Telugu, voice input, speech responses, and stronger grounding controls.

10.7 Deployment and Scalability

Future deployment can migrate from a prototype PostgreSQL database to a production relational database, add stronger authentication, monitoring, backups, role-based access, and secure configuration. A responsive mobile-first interface or dedicated application can also be considered.

REFERENCES

- [1] Open-Meteo, “Open-Meteo Weather API Documentation,” Open-Meteo, 2024.
- [2] Flask Documentation, “Flask Web Development Framework,” Pallets Projects.
- [3] PostgreSQL, “PostgreSQL Documentation,” PostgreSQL Consortium.
- [4] Mozilla Developer Network, “HTML, CSS and JavaScript Web Documentation,” MDN Web Docs.
- [5] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG),” W3C.
- [6] Food and Agriculture Organization of the United Nations, “Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements,” FAO Irrigation and Drainage Paper 56.
- [7] Allen, R. G., Pereira, L. S., Raes, D., and Smith, M., “Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements,” FAO, 1998.
- [8] Zhang, X. et al., “Smart Irrigation and Agricultural Water Management: Digital Approaches for Crop Production,” agricultural water-management literature.
- [9] Netafim, “Digital Farming and Irrigation Management,” Netafim.
- [10] CropX, “Digital Agriculture and Soil Monitoring,” CropX.
- [11] General literature on soil-moisture-based irrigation scheduling and weather-aware irrigation decision support.

APPENDIX A: PACKAGES, TOOLS USED & WORKING PROCESS

A.1 Packages and Tools Used

Table 1.5:TechStack

Component / Tool	Category	Purpose
HTML	Frontend	Page structure and semantic content
CSS	Frontend	Layout, styling, responsive presentation
JavaScript	Frontend	Interaction, validation, dynamic UI behavior
Flask	Backend	Web application framework and server routes
PostgreSQL	Database	Persistent application and activity data
Open-Meteo	External API	Weather and forecast information
LLM	AI Service	Natural-language explanation and chat
Chart Library / JavaScript Charts	Visualization	Historical trends and analytics charts
Git / GitHub	Version Control	Source management and collaboration
Web Browser	Execution	Farmer-facing application environment

A.2 Working Process

- **Requirement identification:** Define the farmer problem and separate the website into focused modules.
- **Interface design:** Plan Dashboard, Weather, Crop Health, Irrigation Planner, Analytics, Myfarms,AI Assistant, and Profile navigation.
- **Backend setup:** Create Flask routes and application structure.
- **Database setup:** Define user, farm profile, activity, soil-moisture, weather-context, and irrigation records.
- **Weather integration:** Connect the selected location to Open-Meteo and map the returned data to the application.
- **Irrigation engine:** Implement validation, averaging of multiple readings, condition analysis, and recommendation output.
- **AI integration:** Send relevant farm context and recommendation information to AI analysis for explanation.
- **Frontend integration:** Display structured results, charts, cards, and actionable recommendations.

APPENDIX B: SOURCE CODE

B.1 Suggested Application Structure

The following logical structure represents the separation of responsibilities used by the AgroMind design. Exact filenames may vary during implementation.

Table 1.6: Application Structure

Area	Representative Files / Responsibility
Frontend	HTML pages, CSS files, JavaScript modules, reusable UI components
Authentication	Registration, login, session handling
Farm Profile	Create, read, update farm information
Weather	Open-Meteo request and weather-data normalization
Irrigation	Input validation, averaging, decision engine, recommendation formatting
Crop Health	Condition scoring and advisory generation
Analytics	Historical aggregation and chart data
AI Assistant	AI request, context preparation, chat history
Database	PostgreSQL schema and persistence functions
Configuration	Environment variables and service credentials

B.2 Irrigation Calculation Logic

A simplified implementation flow is shown below in pseudocode:

```
IF farmer_has_known_moisture:
    final_moisture = entered_moisture
ELSE:
    final_moisture = average(readings)

weather = get_weather(location)
result = irrigation_engine(
    crop, soil_type, farm_size, water_availability,
    final_moisture, weather.temperature,
    weather.humidity, weather.rain_probability,
    weather.rainfall
)

display(result)
```

B.3 Database Initialization — PostgreSQL

```
def get_db():
    os.makedirs(app.instance_path, exist_ok=True)

    connection = PostgreSQL3.connect(DATABASE)
    connection.row_factory = PostgreSQL3.Row

    return connection

def init_db():
    with get_db() as connection:
        connection.executescript("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            email TEXT NOT NULL UNIQUE,
            password_hash TEXT NOT NULL,
            created_at TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS farm_profiles (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER,
            name TEXT,
            location TEXT NOT NULL,
            crop TEXT NOT NULL,
            soil TEXT NOT NULL,
            farm_size REAL NOT NULL DEFAULT 0,
            water TEXT NOT NULL,
            created_at TEXT NOT NULL
        );
        """)
```

B.4 Farm Profile Management

```
@app.route("/api/farms", methods=["GET", "POST"])
@login_required
def farms_api():

    if request.method == "GET":
        with get_db() as connection:
            farms = connection.execute(
                "SELECT * FROM farm_profiles "
                "WHERE user_id = ? ORDER BY id DESC",
                (session["user_id"],)
            ).fetchall()

        return jsonify({
            "success": True,
            "farms": [dict(farm) for farm in farms]
        })

    data = request.get_json(silent=True) or {}

    name = str(data.get("name", "")).strip()
    location = str(data.get("location", "")).strip()
    crop = str(data.get("crop", "")).strip()
    soil = str(data.get("soil", "")).strip()
    water = str(data.get("water", "")).strip()
    farm_size = float(data.get("farm_size", 0))
    with get_db() as connection:
        connection.execute(
            """INSERT INTO farm_profiles
            (user_id, name, location, crop, soil,
            farm_size, water, created_at)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)""",
            (
                session["user_id"], name, location, crop,
                soil, farm_size, water,
                datetime.utcnow().isoformat()
            )
        )
    return jsonify({"success": True}), 201
```

B.5 Irrigation Decision Engine

```

@app.route("/api/irrigation", methods=["POST"])
@login_required
def irrigation():

```

```

    data = request.get_json(silent=True) or {}

```

```

    crop = data.get("crop")
    soil = data.get("soil")
    moisture = float(data.get("moisture", 0))
    temperature = float(data.get("temperature", 0))
    humidity = float(data.get("humidity", 0))
    rainfall = float(data.get("rainfall", 0))
    farm_size = float(data.get("farm_size", 0))
    water = data.get("water")

```

```

    irrigation_required = False

```

```

    if moisture < 30:
        irrigation_required = True

```

```

    if rainfall > 40:
        irrigation_required = False

```

```

    if irrigation_required:
        if temperature > 32:
            recommended_time = "6:00 AM"
        else:
            recommended_time = "7:00 AM"
    else:
        recommended_time = (
            "Delay irrigation" if rainfall > 40
            else "Not required"
        )

```

```

    if irrigation_required:
        if water == "Low":
            duration = 15
        elif water == "Medium":
            duration = 25
        else:

```

```

        duration = 30
    else:
        duration = 0

    if irrigation_required:
        water_quantity = round(farm_size * 600)
    else:
        water_quantity = 0

```

B.6 AI Assistant

```

def generate_ai_explanation(
    crop,
    soil,
    moisture,
    temperature,
    humidity,
    rainfall,
    farm_size,
    water,
    irrigation_required,
    recommended_time,
    duration,
    water_quantity,
    risk
):

    prompt = f"""
    You are AgroMind, an agriculture
    decision-support assistant.

    Crop: {crop}
    Soil Type: {soil}
    Soil Moisture: {moisture}%
    Temperature: {temperature}°C
    Humidity: {humidity}%
    Rain Probability: {rainfall}%
    Farm Size: {farm_size} acres
    Water Availability: {water}

    Irrigation Required: {irrigation_required}

```

Recommended Time: {recommended_time}
Duration: {duration} minutes
Approximate Water Quantity: {water_quantity} L
Risk Level: {risk}

Explain the recommendation to a farmer
in simple language.
""

```
response = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {
            "role": "system",
            "content": "You are a helpful agriculture decision-support assistant."
        },
        {
            "role": "user",
            "content": prompt
        }
    ],
    temperature=0.3,
    max_tokens=250
)

return response.choices[0].message.content
```

B.7 AI Context

The AI Assistant receives only the information required to generate a relevant response, such as:

- Crop type
- Soil type
- Soil moisture
- Weather conditions
- Irrigation recommendation
- Farmer's question

API keys and other sensitive credentials are kept **server-side using environment variables** rather than being exposed in frontend code. AI's documentation also recommends configuring the API key through an environment variable.

This structure keeps the **decision engine responsible for irrigation analysis**, while the AI Assistant is used mainly for **explanation and natural-language interaction**.

Project Documentation Status

This report documents the AgroMind website concept, module design, data flow, decision workflow, implementation structure, outputs, impact considerations, challenges, conclusion, and future development directions. The project is presented as a web-based decision-support prototype for smart irrigation and farm advisory.

Team Members

RONGALI SRIVARUN (24331A4453)

NALLURI SANJANA (24331A4443)

CHINTAPALLI SINDU AKSHITHA (24331A4415)

SUBHOSRI MAHAPATRO (24331A4459)

Project Theme

Agromind — AI-Assisted Smart Irrigation and Farm Advisory System

Core workflow: Farm Data → Weather Data → Soil Moisture → Irrigation Decision Engine → Recommendation → AI Explanation.

The system is intended to support farmers with clear, data-informed irrigation decisions while emphasizing responsible water use and transparent communication.